

第五单元：对象、Date和Math对象

1. 认识对象

1.1. 什么是对象

1.2. 定义对象

1.3. 调用对象

1.4. 添加属性

1.5. Object.assign()

1.6. 遍历对象

1.7. 检查属性

1.8. 删除属性

1.9. JSON属性转换

2. Date对象

2.1. Date 对象获取日期方法

2.2. Date 对象设置日期方法

2.3. Date.now() 日期的静态方法 用于获取时间戳

2.4. 获取时间戳的方法

3. Math对象

3.1. JavaScript Math对象常用方法和属性表格

4. 总结

1. 认识对象

1.1. 什么是对象

- 对象是包含一组无序的键值对的数据集合。
- 键是字符串，而值可以是任意数据类型。
- 在 JavaScript 中，一切皆是对象。

1.2. 定义对象

- 使用对象字面量是最简单的方法：

▼ 定义对象

JavaScript

```
1 var person = {
2   name: "王小明",
3   age: 30,
4   say: function() {
5     console.log('我叫王小明')
6   }
7 };
```

- 也可以使用构造函数创建对象：

▼ 定义对象

JavaScript

```
1 var person = new Object();
2 person.name = "王小明";
3 person.age = 30;
4 person.say = function() {
5   console.log('我叫王小明')
6 };
```

1.3. 调用对象

- 访问属性：使用点符号或方括号。

▼ 调用对象的属性和方法

JavaScript

```
1 var person = {
2   name: "王小明",
3   age: 30,
4   say: function() {
5     console.log('我叫王小明')
6   }
7 };
8 // 访问属性
9 console.log(person.name); // 使用点符号
10 console.log(person['age']); // 使用方括号
11 // 调用方法
12 person.say();
```

1.4. 添加属性

- 可以直接给对象添加新属性

JavaScript

```
1 var person = {  
2   name: "王小明",  
3   age: 30,  
4   say: function() {  
5     console.log('我叫王小明')  
6   }  
7 };  
8  
9 person.address = "上海";
```

1.5. Object.assign()

作用：合并对象, 把后边的所有对象内容合并到第一个对象中, 会改变第一个对象的内容,

若每一个对象的内容都不被改变, 第一个值可放一个空对象{}, 来实现复制对象, 返回一个新对象

合并对象

JSON

```
1 const a1 = { name: 'aaa', b: 'bbb' }  
2 const a2 = { name: '2222', c: 'cccc' }  
3 const a3 = Object.assign(a1, a2, { d: 'ddd' })  
4  
5 console.log(a1) // {name: '2222', b: 'bbb', c: 'cccc', d: 'ddd'}  
6 console.log(a3) // {name: '2222', b: 'bbb', c: 'cccc', d: 'ddd'}
```

JSON

```
1 const a1 = { name: 'aaa', b: 'bbb' }  
2 const a2 = { name: '2222', c: 'cccc' }  
3 const a3 = Object.assign( {}, a1 , a2 , {name : "tom"} )  
4 console.log( a1 ) // { name: 'aaa', b: 'bbb' }  
5 console.log( a3 ) // { name: 'tom', b: 'bbb', c: 'cccc' }  
6 //注意: 属性一致时 会被覆盖
```

1.6. 遍历对象

- 使用for...in循环遍历对象的属性:

```
1 for (var key in person) {  
2   console.log(key + ": " + person[key]);  
3 }
```

对象遍历的顺序 按照属性名转换为数值类型，如果是正整型先遍历正整型，再按照对象的属性添加先后顺序遍历

1.7. 检查属性

对象使用 `in` 检查某个属性是否存在。

通常与`for...in`循环一起使用，或者在单独的表达式中使用来测试某个属性是否存在于对象中。

```
1 let obj = {a: 1, b: 2, c: 3};  
2 console.log('a' in obj); // 输出: true   检查a这个属性是否再obj对象中存在  
3 console.log('d' in obj); // 输出: false  检查d这个属性是否再obj对象中存在
```

1.8. 删除属性

使用 `delete` 来删除对象中的属性

返回值: `boolean` ;

删除成功时，返回`true`

删除不存在的属性时，返回`true`。(删除操作本身是成功的，只是对象没有该属性)

删除不可配置的属性时，返回`false`(如内置对象的属性)

▼ 删除单个

JSON |

```
1  const obj = {
2      user : "张三" ,
3      age : 18
4  }
5  var bol1 = delete obj.name; // 删除不成功 因为name属性不存在 , 但是返回true
6  console.log( obj ) // 输出: { user : "张三" , age : 18 }
7  console.log( bol1 ) // true
8
9  var bol2 = delete obj.user; // 删除成功 因为user属性存在
10 console.log( obj ) // 输出: { age : 18 }
11 console.log( bol2 ) // true
```

▼ 删除多个属性封装函数

JSON |

```
1  var obj = { x : 1 , y : 2 , c : "mom" , "4" : true , a : 5 , b : 10 , e : "hello" }
2  function delAttribute( oObj ,arr ){
3      // 循环遍历拿到每个要删除的属性
4      for( var i = 0 ; i < arr.length; i++ ){
5          // console.log( i , arr[i] ) // 前下标 后项 即属性
6          // 判断当前属性在对象中是否存在 存在删除
7          if(arr[i] in obj) delete obj[arr[i]]
8      }
9      console.log(obj) // {4: true}
10 }
11 //第一个参数是操作的对象 第二个是要删除的key组成的数组对象
12 delAttribute( obj , ["y" , "c" , "e" , "x" , "b" , "a"] )
13 // 如果没有学过数组的同学,可以使用arguments实参对象集合【因为没有办法确定删除几个属性】,
14 // 此时arguments[0]是操作的对象,从下标1开始循环拿到每一个删除的属性key
```

▼ 删除不可配置的属性

JSON |

```
1  var obj = Object.freeze( {x : "hello"} ) //冻结对象,属性不可配置
2  var bol = delete obj.x ;
3  console.log( obj ) // {x : "hello"}
4  console.log( bol ) // false
5
6  //如数学对象Math中内置的属性PI【圆周率】
7  console.log( delete Math.PI ) // false
8  //如用var声明的全局变量也不可以删除
9  var a = "hello-china" // 挂靠window成了window的属性
10 console.log( delete window.a ) // false
```

1.9. JSON属性转换

JSON.parse() 可以将JSON格式的字符串解析成js可以解析的对象

```
1  var json = '{"name": "张三", "age": "18", "gender": "男"}' // json格式字符串
2  //字符串可以通过[下标]访问字符    可以通过length属性访问长度
3  console.log( json[0] ) // 输出: '{'
4  console.log( json.length ) // 输出: 42
5
6  // 将json格式字符串转成对象
7  var userObj = JSON.parse( json )
8  console.log( userObj ) // {"name": "张三", "age": "18", "gender": "男"}
9  console.log( userObj.name ) // 张三    访问
10 userObj.age = 28 ; // 修改
11 console.log( userObj ) // {"name": "张三", "age": 28, "gender": "男"}
```

JSON.stringify()可以将js可以 解析的对象转成json格式的字符串

```
1  var obj = {"name": "张三", "age": 28, "gender": "男"}
2  var jsonObj = JSON.stringify( obj )
3  console.log( jsonObj ) // '{"name": "张三", "age": 28, "gender": "男"}'
```

2. Date对象

- Date对象用于处理日期和时间。
- 创建新的Date实例：

```
1 // 创建当前时间对象，当前电脑显示的时间
2 var date1 = new Date();
3
4 // 传入字符串
5 var date2 = new Date('2023,1,1'); // 2023年1月1日
6 var date3 = new Date('2023/1/1'); // 2023年1月1日
7 var date4 = new Date('2023-1-1'); // 2023年1月1日
8 var date5 = new Date('2023-1-1 12:00:00'); // 2023年1月1日 12:00:00 设置时分秒
9
10 // 传入数字
11 // 月份从0开始（0代表1月，11代表12月）。
12 var date6 = new Date(2023,1,1); // 2023年2月1日
13
14 // 传入时间戳
15 let date7 = new Date(1679982000000);
```

时间戳：从1970年1月1日开始计算的毫秒数。

2.1. Date 对象获取日期方法

方法	描述	返回值
<code>getDate()</code>	获取月份中的某一天（1–31）	数字
<code>getDay()</code>	获取星期几（0–6，0代表星期日）	数字
<code>getFullYear()</code>	获取四位数的年份	数字
<code>getHours()</code>	获取小时（0–23）	数字
<code>getMilliseconds()</code>	获取毫秒（0–999）	数字
<code>getMinutes()</code>	获取分钟（0–59）	数字
<code>getMonth()</code>	获取月份（0–11，0代表1月）	数字
<code>getSeconds()</code>	获取秒数（0–59）	数字
<code>getTime()</code>	获取时间戳，自1970年1月1日以来的毫秒数	数字

2.2. Date 对象设置日期方法

方法	描述	参数
<code>setDate()</code>	设置月份的某一天	1-31
<code>setFullYear()</code>	设置四位数的年份	年份
<code>setHours()</code>	设置小时	0-23
<code>setMilliseconds()</code>	设置毫秒	0-999
<code>setMinutes()</code>	设置分钟	0-59
<code>setMonth()</code>	设置月份	0-11 (0代表1月)
<code>setSeconds()</code>	设置秒数	0-59
<code>setTime()</code>	设置自1970年1月1日以来的毫秒数	毫秒数

2.3. Date.now() 日期的静态方法 用于获取时间戳

2.4. 获取时间戳的方法

即: 自1970年1月1日以来的毫秒数

1. `date.getTime()`
2. `date.valueOf()`
3. `new Date() * 1`
4. `+new Date()`
5. `Date.now()`

```

1  var date = new Date();    // 创建日期对象 即 实例化了一个日期对象
2  // 从创建的日期对象中 使用getTime 或 valueOf 方法可以获取
3  console.log( date.getTime() ) // 1749259071572
4  console.log( date.valueOf() ) // 1749259071572
5  // 隐式转换
6  console.log( date * 1 , new Date() * 1 ) // 1749259071572 1749259071572
7  console.log( +date , +new Date() )    // 1749259071572 1749259071572
8  // Date在构造函数本身访问now方法
9  console.log( Date.now() )    // 1749259071572

```

3. Math对象

- Math提供了一系列数学常数和函数，不需要 new， 可以直接使用。
- 示例：

Math对象

JavaScript

```

1  console.log(Math.PI); // 圆周率π
2  console.log(Math.abs(-10)); // 10 获取绝对值
3  console.log(Math.random()); // 0到1之间的随机数

```

3.1. JavaScript Math对象常用方法和属性表格

类型	名称	描述	示例
属性	Math.PI	圆周率 π 的值	Math.PI
属性	Math.E	自然对数的底数e的值	Math.E
方法	Math.abs(x)	返回x的绝对值	Math.abs(-10)
方法	Math.ceil(x)	对x向上取整	Math.ceil(9.1)
方法	Math.floor(x)	对x向下取整	Math.floor(9.9)
方法	Math.round(x)	将x四舍五入为最近的整数	Math.round(4.5)
方法	Math.max(x1, x2, ..., xn)	返回一组数中的最大值	Math.max(10, 20, 30)
方法	Math.min(x1, x2, ..., xn)	返回一组数中的最小值	Math.min(10, 20, 30)

方法	Math.pow(x, y)	返回x的y次幂	Math.pow(2, 3)
方法	Math.sqrt(x)	返回x的平方根	Math.sqrt(16)
方法	Math.random()	返回0到1之间的伪随机数	Math.random()
方法	Math.sin(x)	返回角x的正弦值	Math.sin(Math.PI / 2)
方法	Math.cos(x)	返回角x的余弦值	Math.cos(Math.PI)
方法	Math.tan(x)	返回角x的正切值	Math.tan(Math.PI / 4)
方法	Math.asin(x)	返回x的反正弦值	Math.asin(1)
方法	Math.acos(x)	返回x的反余弦值	Math.acos(1)
方法	Math.atan(x)	返回x的反正切值	Math.atan(1)
方法	Math.log(x)	返回x的自然对数	Math.log(Math.E)

4. 总结

1. 创建对象的两种方式，子面量和new关键字。
2. 调用对象的属性和方法，给对象添加新属性和方法。
3. for in 遍历对象。
4. 日期对象的常用方法。
5. 数学对象的常用方法。